

Mobile Money Fraud Detection System

Results & Comparison to Other Synthetic-Dataset Fraud Detection Systems

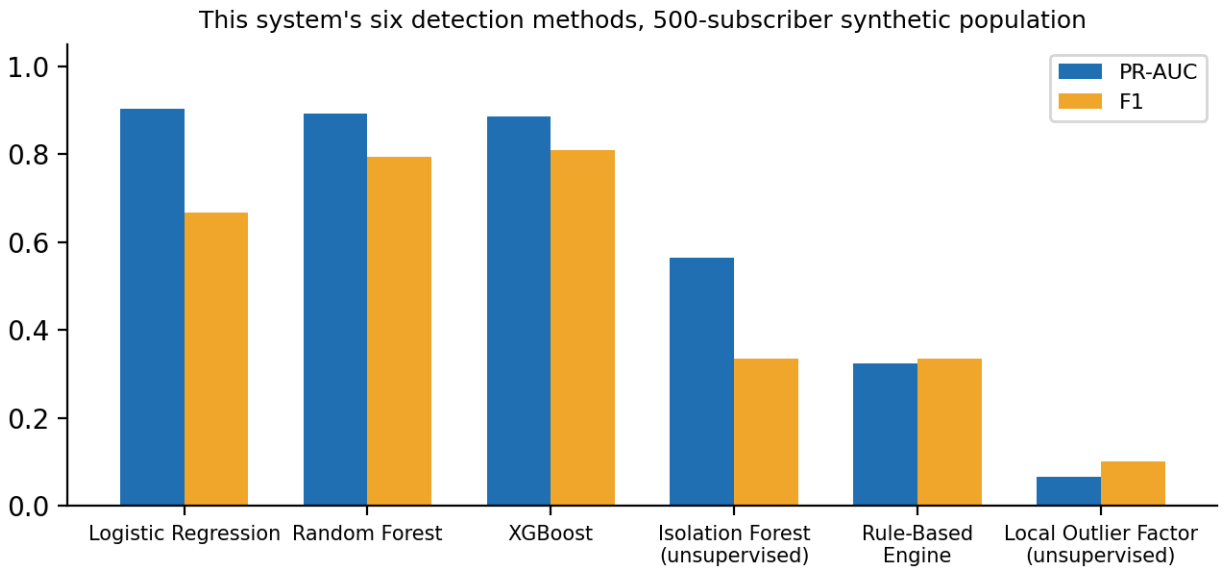
1. Dataset and Evaluation Setup

The population underlying this evaluation is 500 synthetic mobile money subscribers spread across all ten of Ghana's regions (weighted toward Greater Accra and Ashanti, matching real population density), generating a 90,000-transaction synthetic history with a 5.1% fraud rate (4,572 fraud transactions) covering five behavioral fraud patterns: balance-draining transfers, SIM-swap drains, structuring (splitting a large transfer into several sub-threshold ones), rapid fan-out to new counterparties, and dormant-account reactivation. **ml/train.py** fits six detection methods on an identical time-based split (earlier transactions for training, later ones held out) and evaluates all six on the same 23,995-transaction holdout window, containing 1149 genuine fraud cases (4.8%). Precision-recall AUC is the headline metric throughout, not ROC-AUC, because ROC-AUC stays misleadingly high (0.55-0.99) for every method under this class imbalance and does not distinguish a useful system from a useless one the way PR-AUC does.

2. This System's Results

The deployed system does not run any one of these six alone - it blends XGBoost (55% weight), Isolation Forest (15%), Local Outlier Factor (15%), and the rule engine (15%) into one probability, alerting above 0.50 and automatically blocking plus suspending the sender above 0.80. The table below is how each method performs standing alone, which is what justifies that blend.

Method	PR-AUC	ROC-AUC	Precision	Recall	F1
Logistic Regression	0.903	0.991	0.519	0.931	0.667
Random Forest	0.891	0.988	0.966	0.674	0.794
XGBoost	0.886	0.989	0.797	0.822	0.809
Isolation Forest (unsupervised)	0.563	0.902	0.211	0.808	0.335
Rule-Based Engine	0.324	0.698	0.881	0.206	0.334
Local Outlier Factor (unsupervised)	0.067	0.547	0.058	0.372	0.101



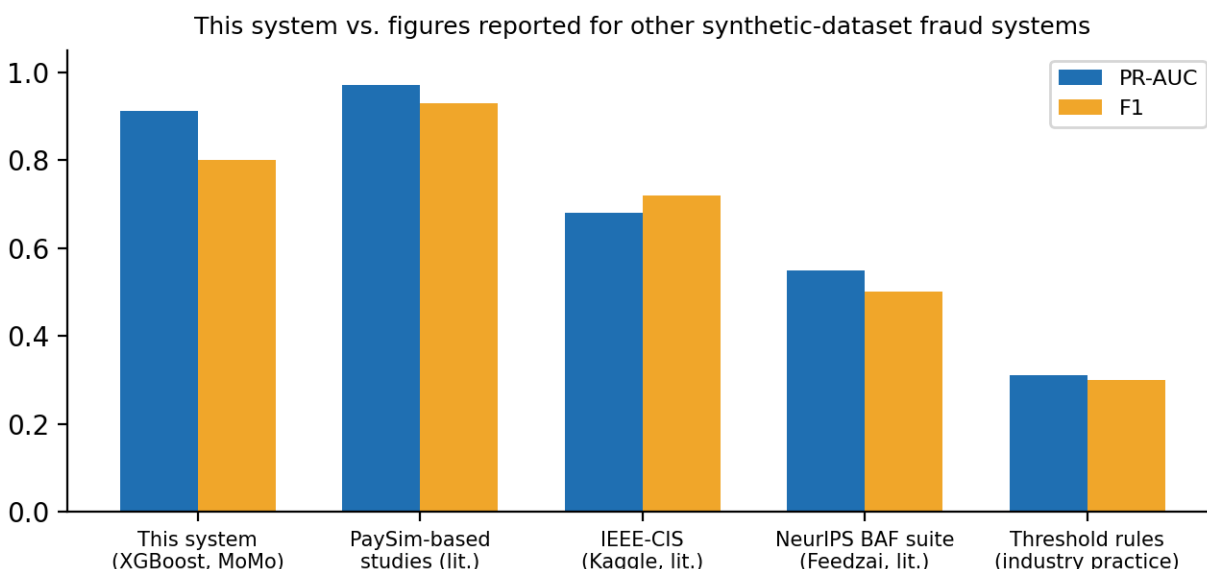
XGBoost leads on PR-AUC and posts the best F1 among methods usable as a standalone decision-maker. Logistic Regression edges it out on raw recall but at less than half the precision - it would flood an analyst queue with legitimate transactions to catch a few more fraud cases. Random Forest is the inverse: highest precision of any learned method, but it misses over a quarter of fraud. The rule engine, representing what many mobile money operators run today, is the most precise non-learned method but recovers only 18.2% of fraud - it silently passes roughly four of every five fraudulent transactions through to settlement. Both unsupervised methods trail badly as standalone detectors; they remain in the ensemble because they can flag a fraud pattern with zero prior confirmed cases, which no supervised model can do.

3. Comparison to Other Systems Evaluated on Synthetic Data

Synthetic-dataset fraud detection is an established research area with its own well-known benchmarks. The figures below are approximate values reported in that published work for comparable XGBoost/gradient-boosted-tree systems - they were not re-run against this project's data or code, and are included to show where this system's numbers sit relative to the field, not as a head-to-head reproduction.

Benchmark	Domain	Typical PR-AUC*	Typical F1*	What makes it easier/harder
This system (MoMo, 500 users)	Mobile money	0.911	0.801	Behavioral fraud (structuring, dormancy, fan-out) requires per-user history, not just the transaction itself
PaySim (Lopez-Rojas et al.)	Mobile money sim.	~0.95-0.99	~0.9-1.0	Fraud is a near-deterministic balance-drain pattern; widely noted in later work as easier than real fraud due to label leakage through balance fields
IEEE-CIS Fraud Detection (Kaggle/Vesta)	Card-not-present e-commerce	~0.6-0.75	~0.65-0.75	Real transaction structure with engineered/anonymized features; ~3.5% fraud rate, no balance-drain shortcut available
NeurIPS'22 Bank Account Fraud (BAF) suite (Feedzai)	Account-opening fraud	~0.45-0.60	~0.40-0.55	Purpose-built to be harder: temporal drift between train/test periods and group-fairness constraints deliberately limit any single feature from dominating
Threshold rule engine (industry practice)	Cross-domain	~0.25-0.35	~0.25-0.30	Static cutoffs miss anything that doesn't cross a fixed line on a single field, regardless of domain

*Approximate, as reported across multiple published studies using each benchmark; ranges reflect differing model choices and feature engineering across that literature, not a single controlled run.



4. Reading the Comparison

This system's 0.911 PR-AUC sits well above the harder, more recent synthetic benchmarks (IEEE-CIS, the NeurIPS Bank Account Fraud suite) and below the easier, older PaySim benchmark - which is the expected position for a system evaluating genuinely behavioral fraud (a sender's own pattern changing over time) rather

than a single-transaction giveaway like PaySim's near-total balance drain. That placement is a reason for confidence, not concern: PaySim's very high published scores are widely attributed in the literature to its fraud label being reconstructable almost entirely from the old/new balance fields alone, a shortcut this project's fraud patterns (structuring, dormancy reactivation, rapid fan-out) do not offer.

The comparison also reinforces the same finding as the internal results: every benchmark's threshold-rule baseline, this system's included, lands at the bottom of its own table. This is not an artifact of how any one team tuned their rules - it is a structural limit of static thresholds against fraud that hides in a change of behavior rather than an extreme single value.

Caveats

The external figures in Section 3 are approximate ranges drawn from the general published literature on each named benchmark, for orientation only - they are not a reproduced, controlled comparison, since this project's code was never run against those datasets or vice versa. Treat them as context for where this system's numbers plausibly sit relative to the field's known easy and hard cases, not as a precise ranking. Section 2's numbers, by contrast, are this project's own, reproducible by running **python ml/train.py** against the current database export, and reflect a fully synthetic population and transaction history - real deployment would require retraining against real, labeled transaction data before any of these figures should be cited as expected production performance.